

Minimax Chess

Ayush Pai, Richard Tan, Luke Allen, Mihika Gupta, Nat Heddaeus
Department of Computer Science
University of North Carolina
Chapel Hill, NC 27599

November 20, 2025

Abstract

Adversarial searches are often used by chess engines to identify the optimal course of actions to take over the course of a game. In this study, we aim to replicate existing chess algorithms using a minimax implementation of the computer player. We additionally implement three add-ons: (1) an LLM referee which analyzes the game state after 10 total moves, (2) an evaluation bar visualizing this computer assessment, and (3) a custom "super-knight" piece outside of the standard set. We find that our algorithm is effectively able to identify winning moves in checkmate-in-one and checkmate-in-two scenarios.

1 Introduction and Motivation

Adversarial searches, such as the minimax algorithm and Monte-Carlo Tree Search, are used in competitive games where multiple agents have different goals. For zero-sum games in particular, adversarial searches can be employed by AIs to determine the best course of action given the possible moves of the players (GeeksForGeeks, 2025). For this reason, adversarial searches are often used by chess engines in the creation of computer players.

Our main goal was to create a system that not only plays chess using adversarial search, but also demonstrates core course ideas such as heuristic evaluation, zero-sum modeling, and intelligent decision-making under uncertainty.

2 Related Work

2.1 Chess Engines

Chess engines traditionally use a modified minimax search, in which branches are modified to reduce the costs of a large branching factor. Stockfish, for example, is an open-source chess engine that uses aggressive alpha-beta pruning to avoid searching unpromising branches, which allows for deeper, faster searches at the

cost of potential moves that would have been evaluated in a full minimax search (Stockfish). For minimax algorithms which use alpha-beta pruning, the algorithm is always guaranteed to find the optimal solution, thus the computer player can quickly and always find the best maximizing or minimizing move given the current game state. For this reason, Stockfish is a popular choice among online chess sites in creating computer player AIs.

2.2 LLM Analysis

While Large Language Models (LLMs) lack the calculative capabilities required of stronger chess engines, and instead require that they be trained on existing data, which can be drawn from chess analysis engines to simulate the evaluation capabilities of an advanced algorithm. For example, in the paper, “Complete Chess Games Enable LLM Become A Chess Master”, Zhang et al. train an LLM coined ChessLLM using open-llama-3B, which is part of an open-source project that offers a training pipeline for building LLMs (Open-Llama). Using large scale datasets from Stockfish for their models, they find that ChessLLM is able to complete a game of chess (Zhang et al., 2025). This suggests that LLMs have the potential to serve as useful tools to chess.

2.3 Evaluation Bars

Using deep analysis of their respective algorithms, advanced chess engines such as Stockfish and Komodo are able to assign numerical values to material or positional advantages. For example, for the minimax algorithm, a positive value indicates that white, the maximizing player, has the larger perceived advantage, while a negative value indicates that the advantage belongs to black (Chess.com). The evaluation bar is a visual indicator of this evaluation score.

2.4 Custom Pieces

Chess variants, sometimes referred to as Fairy Chess, involve changes to the board, pieces, or moves. For these variants, chess-engines can be derived from existing engines to support the modified game. Stockfish, for example, offers Fairy-Stockfish, another of which supports variants such as Xiangqi, Crazyhouse, and Atomic (Stockfish). Chess.com offers similar variants.

3 Methods

3.1 Base Chess Algorithm

Our base algorithm utilizes an iterative deepening loop paired with the minimax algorithm to create a chess game played against a computer player. Given the time and computing-power restraints of this experiment, the minimax algorithm is pruned to a depth of four. For the purposes of this experiment, depth can be defined as the number of moves the AI looks ahead of itself and the player.

Additionally, the human player is constrained to white pieces, and the computer player to black pieces.

In our minimax algorithm, the computer player seeks to minimize the total value of the pieces on the board. This value is our evaluation score. The score is given an initial value of zero, but can be swayed positive or negative depending on which player holds the more valuable pieces. A positive score corresponds to a white-side advantage, while a negative score corresponds to a black-side advantage.

3.2 LLM Referee

After every ten moves, meaning five from each player, the LLM referee will analyze the current game state using the evaluation score provided by the minimax algorithm. It provides the user with its own analysis of the game state.

3.3 Evaluation Bar

The evaluation bar uses the evaluation score to determine the percentages of the bar which are white and black. At game start, these percentages are equal. An increase in the percentage of one color corresponds to the advantage shifting in that player’s favor.

3.4 Custom Piece

We additionally implement a “super-knight” piece, which is a variant of the traditional knight piece. It can move in the same directions as the traditional knight, but instead moves equally horizontally and vertically, at two spaces per axis. It is valued at 8 points, 5 points more than the traditional knight piece.

4 Experiments and Results

To test the reliability of our base algorithm, we provided the engine with several checkmate puzzles requiring less than or equal to 4 moves (engine depth). For convenience of testing, we flip the colors of the player and computer so that the engine can take on the perspective of a chess player and seek the solution to the puzzle.

In the Figure 1, we have a checkmate in one puzzle where our algorithm evaluates the white pieces’ position. There’s a black queen on D7 that can be immediately captured by the rook on D1. However, our algorithm is not greedy as it expands all of the nodes (possible sequence of moves) within 4 moves. Upon evaluation via minimax, the engine ignores the queen that would’ve given the most immediate increase in point evaluation (+9), but instead moves the knight to F6 to attack the king. This move immediately gains 0 points but minimizes the point evaluation of the opponent because it can do nothing to save the game facing this move from the engine. Ultimately, knight to F6 guarantees a checkmate in the second move, improving the position to +10000.



Figure 1: Checkmate in One



Figure 2: Checkmate in Two

In the second experiment, we intend to demonstrate the defensive accuracy of our engine. In this position, black is threatening a checkmate on Q2. Therefore, as white, the engine must defend that square to continue the game. Upon evaluation, the engine correctly avoids capturing the black queen on E5 despite the move having the most point increase in one move. It evaluates all of the possible move (nodes in the search tree) for the current player and the opponent to find the best alternative move. This move avoids the checkmate in two for the opponent and correctly defends the position to continue the game.

5 Conclusion

The minimax algorithm is an effective strategy employed by many chess engines in the creation of their computer players. In this study, we sought to replicate existing chess engines using this algorithm at a depth of four. We additionally implement an LLM referee, an evaluation bar, and a custom "super-knight" piece. We find that our algorithm is effectively able to identify winning moves in checkmate-in-one and checkmate-in-two scenarios.

References

- [1] “Adversarial Search Algorithms in Artificial Intelligence (AI).” GeeksforGeeks, GeeksforGeeks, 23 July 2025, www.geeksforgeeks.org/artificial-intelligence/adversarial-search-algorithms/.
- [2] “Computer Evaluation Numbers on Chess.Com (like +2.25).” Chess.Com Help Center, support.chess.com/en/articles/8379354-computer-evaluation-numbers-on-chess-com-like-2-25. Accessed 18 Nov. 2025.
- [3] “Deep Blue.” IBM, www.ibm.com/history/deep-blue. Accessed 19 Nov. 2025.
- [4] “Fairy-Stockfish.” fairy-stockfish.github.io/. Accessed 18 Nov. 2025.
- [5] “Frequently Asked Questions.” Frequently Asked Questions | Stockfish Docs, official-stockfish.github.io/docs/stockfish-wiki/Stockfish-FAQ.html. Accessed 18 Nov. 2025.
- [6] “Komodo - Chess Engines.” Chess.Com, www.chess.com/terms/komodo-chess-engine. Accessed 18 Nov. 2025.
- [7] s-JoL. “S-JoL/Open-Llama: The Complete Training Code of the Open-Source High-Performance Llama Model, Including the Full Process from Pre-Training to RLHF.” GitHub, github.com/s-JoL/Open-Llama. Accessed 18 Nov. 2025.
- [8] Zhang, Yinqi, et al. "Complete Chess Games Enable LLM Become A Chess Master." arXiv preprint arXiv:2501.17186 (2025).